



CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework Milestone 2

2024/25 Spring

Student Name: Krishal Koirala

London Met ID: 24046399

College ID: np01nt4a240032

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count:2811

Project File Links:

Onedrive Drive Link:	KrishalKoirala_24046399((python).docx
----------------------	---

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Krishal Koirala1.docx

 Islington College,Nepal

Document Details

Submission ID

trn:oid::3618:95736545

Submission Date

May 14, 2025, 4:03 AM GMT+5:45

Download Date

May 14, 2025, 4:15 AM GMT+5:45

File Name

Krishal Koirala1.docx

File Size

10.8 KB

25 Pages

1,406 Words

7,796 Characters



12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **18 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 12%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 18 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0% Internet sources**
- 0% Publications**
- 12% Submitted works (Student Papers)**

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	islingtoncollege on 2025-05-12	2%
2	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-04-03	2%
3	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-03-14	1%
4	Submitted works	Kaplan International Colleges on 2024-02-25	1%
5	Submitted works	University of Northampton on 2019-05-12	1%
6	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2021-06-18	1%
7	Submitted works	University of Wales Institute, Cardiff on 2024-09-30	<1%
8	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-06-17	<1%
9	Submitted works	Wayne State University on 2024-10-17	<1%
10	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2022-06-10	<1%

11	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-12-29		<1%
12	Submitted works	
University of Northumbria at Newcastle on 2024-09-02		<1%

Table of Contents

1. Introduction	1
1.1 Brief information about the project	1
1.2 Aims and objectives	1
1.3 Technologies used	2
2.Data Structures.....	3
2.1.Algorithm	3
2.2 Flowchart	4
2.3 Pseudo Code	5
2.4 Data Structures.....	7
3. Program Implementation	8
3.1 Purchase and Sell.....	9
3.2 Creation of txt file.....	10
3.3 Bill of txt file.....	10
3.4 End of the program	11
4 Testing	12
4.1 Test 1 Error Handling.....	12
4.2 Test 2 Purchase and Sale Validation.....	13
4.3 Test 3 Purchase Process.....	14
4.4 Test 4 Sales process	15
4.5 Test 5 Stock Updates	17
Bibliography	27

List of Figures

Figure 1 Data Structs (List)	7
Figure 2 Data Structure(Strings)	8
Figure 3 Data Structure (Integer)	8
Figure 4 Program Implementation(Sale)	9
Figure 5 Program Implementation(Purchase)	9
Figure 6 Creation of txt file	10
Figure 7 Bill in txt file	10
Figure 8 End of the program	11
Figure 9 Testing 1 SS.....	12
Figure 10 Testing 2 SS.....	13
Figure 11Testing 3 SS.....	14
Figure 12 Testing 4 Selling 1st product.....	15
Figure 13 Testing 4 Selling 2nd Product	15
Figure 14 Testing 4 Bill of both products.....	16
Figure 15 Testing 5 Before selling product.....	17
Figure 16 Testing 5 while selling product	17
Figure 17 Testing 5 After Selling product	17
Figure 18 Testing 5 while purchasing product.....	18
Figure 19 Testing 5 After purchasing product	18

List of Tables

Table 1 Test 1 Error Handling	12
Table 2 Test 2 Purchase and Sale Validation	13
Table 3 Test 3 Purchase Process	14
Table 4 Sales process.....	15
Table 5 Stock Updates	17

1. Introduction

1.1 Brief information about the project

The "WeCare Wholesale Inventory System" is a Python-based application developed to streamline and automate the management of skincare product inventory in a wholesale business setting. It aims to simplify tasks such as restocking items, processing customer sales, and tracking current stock levels.

The system provides an intuitive and user-friendly interface for administrators to interact with inventory data. By incorporating robust error handling, persistent file storage, and transaction-based billing, the application ensures operational accuracy and improves overall efficiency in wholesale operations.

Furthermore, the system generates detailed invoice files for every sale and restock operation, logs updated product quantities, and implements bonus product logic (e.g., 1 free item per 3 sold). Altogether, this software ensures structured, reliable, and effective control over day-to-day inventory activities.

1.2 Aims and objectives

Aims

A wholesale management system would replace a manual stock system with an electronic one. The system would automatically track stocks, update after each sale, and prompt restocking when required. The system would also sort sales data in terms of categorizing it in order to be able to narrate trends and top-selling items, hence being able to control inventory quicker, clearer, and more effectively. This ensures smooth operations and accurate decision-making.

Objectives

- Have reading and writing of product information via text files.
- Have stock level updates automatically after every transaction.
- Have mark-up and discount as part of pricing logic.
- Generate accurate invoices for both vendor restocking and customer sales in.txt format.

1.3 Technologies used

Programming Language – Python

Tools – IDLE

2.Data Structures

2.1.Algorithm

Step 1: Start the system

Step 2: Display shop header and welcome message

Step 3: Show main menu with the following options:

- Press 1 to **Sell product to customer**
- Press 2 to **Restock products from manufacturer**
- Press 3 to **Exit the system**

Step 4: Take input from user and store it as user_choice

Step 5:

- If user_choice == 1, go to Step 6
- Else if user_choice == 2, go to Step 14
- Else if user_choice == 3, go to Step 18
- Else, show "Invalid choice" and go back to Step 3

Step 6: Ask and input customer name

Step 7: Ask and input customer phone number

Step 8: Show available products from the inventory

Step 9: Ask for product ID and quantity to sell

Step 10:

- Validate product ID and stock
- Apply free item logic: for every 3 sold, give 1 free
- Calculate total and add to sale list
- Repeat if customer wants to add more items

Step 11:

- Calculate shipping charge (10% if total < 1000)
- Generate invoice file with customer and product details
- Save updated stock back to file

Step 12: Go back to Step 3

Step 13: Show available products

Step 14: Ask for product ID and quantity to add

Step 15: Validate product ID and quantity

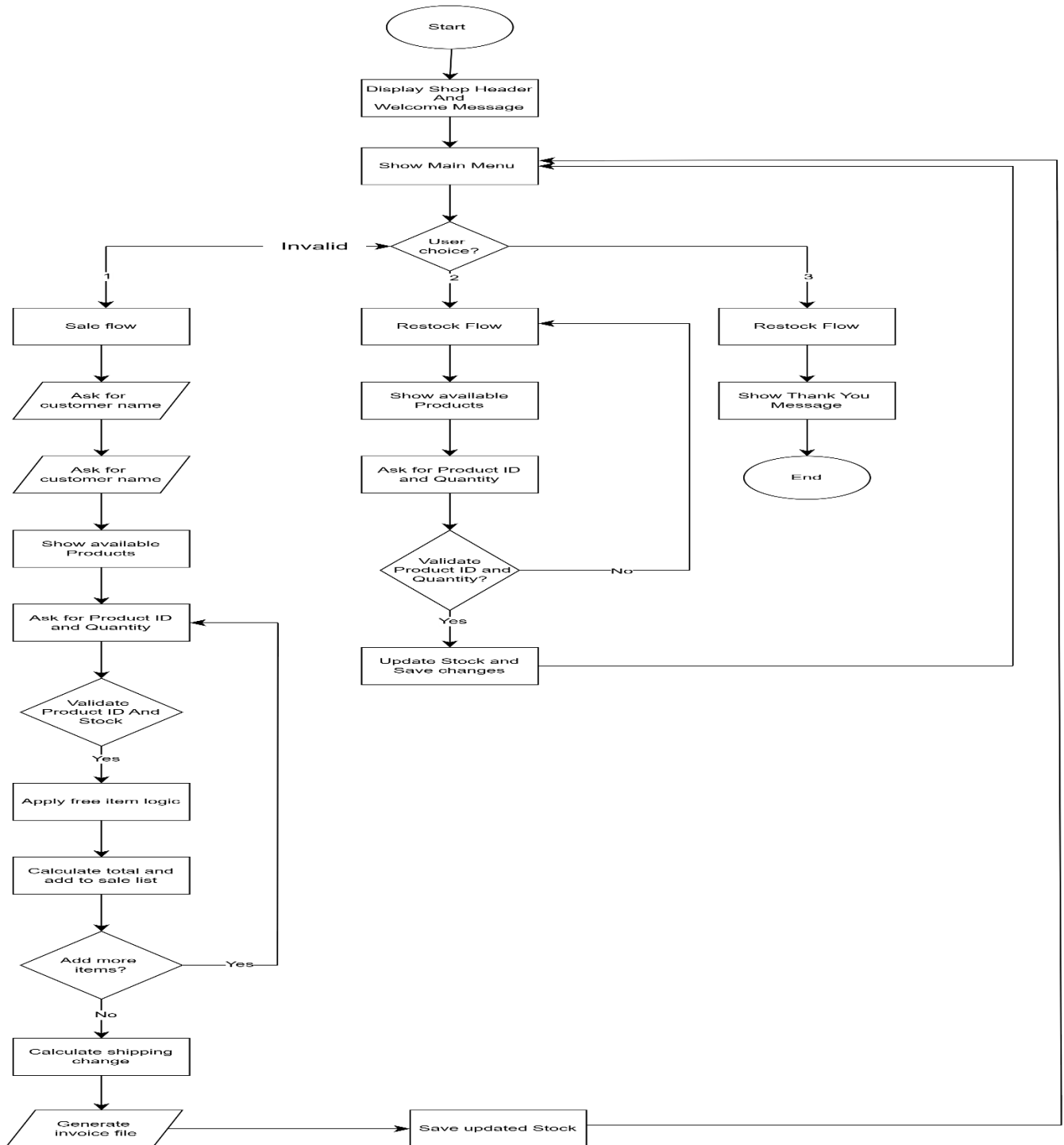
Step 16: Update stock and save changes to file

Step 17: Return to Step 3

Step 18: Show "Thank you for using our system"

Step 19: End/Exit the program

2.2 Flowchart



2.3 Pseudo Code

1. Main Program

```
IMPORT MODULE read_product_data FROM read
IMPORT MODULE handle_sale, handle_restock FROM operations
```

```
Def function main():
```

```
    DISPLAY shop header information
```

```
    CALL function read_product_data() → STORE result in products
```

```
    If products IS EMPTY:
```

```
        DISPLAY "No product data found"
```

```
    End function main
```

```
While True:
```

```
    DISPLAY main menu options (1. Sell, 2. Restock, 3. Exit)
```

```
    INPUT user_choice
```

```
    If user_choice == 1:
```

```
        CALL function handle_sale(products)
```

```
    ElseIf user_choice == 2:
```

```
        CALL function handle_restock(products)
```

```
    ElseIf user_choice == 3:
```

```
        DISPLAY "Thank you for using our system."
```

```
        BREAK from loop
```

```
    Else:
```

```
        DISPLAY "Invalid input. Please choose 1, 2 or 3."
```

```
End While
```

```
End function main
```

```
Call function main()
```

2. Read Product Data

```
Def function read_product_data(filename):
```

CREATE empty dictionary products

Try:

 OPEN filename for reading

 For each line in file:

 SPLIT line by ","

 STORE values in products dictionary

Except FileNotFoundError:

 DISPLAY "Error: File not found"

Return products

End function read_product_data

3. Write Product Data

Def function write_product_data(products, filename):

 Try:

 OPEN filename for writing

 For each product in products:

 WRITE product info to file

 Except:

 DISPLAY error message

End function write_product_data

4. Handle Sale

Def function handle_sale(products):

 INPUT customer_name

 INPUT customer_phone

 CREATE empty list sold_items

 SET total_amount = 0

 While True:

 CALL function display_product_table(products)

 INPUT product_id, quantity

 If product_id is invalid OR quantity <= 0:

 DISPLAY error

 CONTINUE

 Calculate free_items = quantity // 3

 Check if stock is sufficient

 Update stock

 Calculate total_cost

```

    Append sale info to sold_items list
    total_amount = total_amount + total_cost

    Ask if more items to add
    If not, BREAK
End While

If total_amount < 1000:
    shipping_cost = 10% of total_amount
Else:
    shipping_cost = 0

grand_total = total_amount + shipping_cost
Generate invoice file with sale details
Call write_product_data(products)
End function handle_sale

```

The system uses various data structures in Python to handle and process product data

2.4 Data Structures

1 List

This Data Structures is used to store the lines of data read from the text file

- file = open("products.txt", "r")
- data = file.readlines()
- file.close()

```

--
file = open(filename, "r")
lines = file.readlines()

```

Figure 1 Data Structure (List)

2 Dictionary

This Data structures are used to store product info where each product has a unique ID as the key and product details as the value.

- d = {1:["Ultra Sunscreen ","Nuetrogena ","399"," 1199"," USA"]}

3 Strings

This Data structures is used for handling text data like products names, customer names, and invoice content

➤ product_name = "Anthelois"

```
customer_name = input("Enter customer name: ")
if customer_name == "":
    print("Customer name cannot be empty.")
```

Figure 2 Data Structure(Strings)

4 Integer

For storing numeric data like price, quantity

➤ int(d[product_id][2])

```
customer_phone = int(input("Enter customer phone number: "))
```

Figure 3 Data Structure (Integer)

3. Program Implementation

The program consists of several modules :

main.py	– Entry point with main menu loop
operations.py	– Contains sales and restock handling functions
read.py	– Handles reading product data from file
write.py	– Handles writing product data to file

Key features implemented:

- Product sales with "buy 2 get 1 free" promotion
- Automatic 10% shipping charge for orders under Rs. 1000
- Detailed invoice generation with timestamps
- Inventory tracking with immediate updates
- Input validation and error handling

3.1 Purchase and Sell

Sell

```

                                WeCare Wholesale
                                Kamalpokhari, Kathmandu | Phone No: 9812345670
=====
                                Welcome to the system Admin! I hope you have a good day ahead!
=====

-----
                                MAIN MENU
-----
1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
-----
Enter your choice (1, 2, or 3): 1
Enter customer name: Krishal
Enter customer phone number: 9845713016

=====
                                AVAILABLE PRODUCTS
=====
ID      Name                    Brand                Qty    Price    Origin
-----
1       Anthelois               La Roche-Posay       583    5196    France
2       Nivea SunProtect            Nivea                 538    1998    Germany
3       Ultra Sunscreen            Nuetrogena            399    2398    USA
-----

Enter product ID to sell: 1
Enter quantity to sell: 10
Add another item? (yes/no): no
=====

```

Figure 4 Program Implementation(Sale)

Purchase

```

1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
-----
Enter your choice (1, 2, or 3): 2

=====
                                AVAILABLE PRODUCTS
=====
ID      Name                    Brand                Qty    Price    Origin
-----
1       Anthelois               La Roche-Posay       570    5196    France
2       Nivea SunProtect            Nivea                 538    1998    Germany
3       Ultra Sunscreen            Nuetrogena            399    2398    USA
-----

Enter product ID to restock: 1
Enter quantity to add: 20
=====

```

Figure 5 Program Implementation(Purchase)

3.2 Creation of txt file

Products.txt

```
Anthelois      ,La Roche-Posay,590, 2598, France
Nivea SunProtect,Nivea      ,538, 999 , Germany
Ultra Sunscreen ,Nuetrogena  ,399, 1199, USA
```

Figure 6 Creation of txt file

3.3 Bill of txt file

```
=====
                        WeCare Wholesale
                Kamalpokhani, Kathmandu | Phone: 9812345670
                        SALES INVOICE
=====
Customer Name : Krishal
Phone Number  : 9845713016
Date          : 2025-05-14 01:50:57.527725
=====
No  Product Name      Qty  Free  Price  Total
-----
1  Anthelois          10    3   5196   51960
=====
Subtotal       : Rs. 51960
Grand Total    : Rs. 51960
=====
                        Thank you for shopping with us!
=====
```

Figure 7 Bill in txt file

3.4 End of the program

```
-----  
                                MAIN MENU  
-----  
1. Sell product to customer  
2. Restock products from manufacturer  
3. Exit the system  
-----  
Enter your choice (1, 2, or 3): 3  
  
Thank you for using the system. Goodbye Admin!
```

Figure 8 End of the program

4 Testing

4.1 Test 1 Error Handling

Objective	To check the implementation of try and expect
Action	Run the main file and input non-numeric or not included number in the program
Expected Result	An error should be created and displayed without crashing the file
Actual Result	An error is created and displayed without crashing the file.
Conclusion	The test was successful.

Table 1 Test 1 Error Handling

```

                                WeCare Wholesale
                                Kamalpokhari, Kathmandu | Phone No: 9812345670

=====
                                Welcome to the system Admin! I hope you have a good day ahead!
=====

-----
                                MAIN MENU
-----
1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
-----
Enter your choice (1, 2, or 3): 1
Enter customer name:
Customer name cannot be empty.
Enter customer name: Krishal
Enter customer phone number:
Invalid phone number. Digits only.
Enter customer phone number: 98saas
Invalid phone number. Digits only.
Enter customer phone number: 9845713016

```

Figure 9 Testing 1 SS

4.2 Test 2 Purchase and Sale Validation

Objective	To verify negative and non-existent product handling.
Action	Enter negative quantity or invalid product ID
Expected Result	Error message, request for valid input
Actual Result	An error is created and displayed without crashing the file.
Conclusion	The test was successful

Table 2 Test 2 Purchase and Sale Validation

Welcome to the system Admin! I hope you have a good day ahead!					
=====					

MAIN MENU					

1. Sell product to customer					
2. Restock products from manufacturer					
3. Exit the system					

Enter your choice (1, 2, or 3): 2					
=====					
AVAILABLE PRODUCTS					
=====					
ID	Name	Brand	Qty	Price	Origin

1	Anthelouis	La Roche-Posay	590	5196	France
2	Nivea SunProtect	Nivea	538	1998	Germany
3	Ultra Sunscreen	Nuetrogena	399	2398	USA

Enter product ID to restock: -2					
Invalid product ID.					

MAIN MENU					

1. Sell product to customer					
2. Restock products from manufacturer					
3. Exit the system					

Enter your choice (1, 2, or 3): 4					
Invalid option. Please choose between 1, 2, or 3.					

MAIN MENU					

1. Sell product to customer					
2. Restock products from manufacturer					
3. Exit the system					

Enter your choice (1, 2, or 3):					

Figure 10 Testing 2 SS

4.3 Test 3 Purchase Process

Objective	Verify complete purchase process
Action	Purchase multiple products with valid inputs
Expected Result	Inventory updated ; invoice generated
Actual Result	Correct inventory deduction , invoice file created
Conclusion	The test was successful

Table 3 Test 3 Purchase Process

Buying Stocks

```

MAIN MENU
-----
1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
-----
Enter your choice (1, 2, or 3): 2

=====
AVAILABLE PRODUCTS
=====
ID   Name           Brand           Qty   Price   Origin
-----
1    Anthelois       La Roche-Posay  590   5196   France
2    Nivea SunProtect Nivea           538   1998   Germany
3    Ultra Sunscreen  Nuetrogena      399   2398   USA

Enter product ID to restock: 3
Enter quantity to add: 500

=====
WeCare Wholesale
Kamalpokhari, Kathmandu | Phone: 9812345670
RESTOCK RECEIPT
=====
Product Restocked:
=====
ID   Product Name      Brand           Added   New Stock
-----
3    Ultra Sunscreen    Nuetrogena      500     899

Stock updated successfully.

Restock receipt saved as: restock_20250514_023800.txt

```

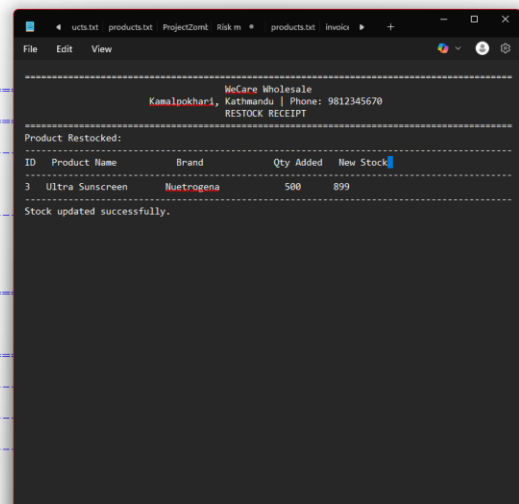


Figure 11 Testing 3 SS

4.4 Test 4 Sales process

Objective	File generation of purchase of products
Action	Should choose 2 product and buy by the user
Expected Result	Inventory updated, receipt generated
Actual Result	Correct inventory addition, receipt file created
Conclusion	The test was successful

Table 4 Sales process

Selling first product

```

=====
                          MAIN MENU
=====
1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
=====
Enter your choice (1, 2, or 3): 1
Enter customer name: Krishal
Enter customer phone number: 9845713016

=====
                          AVAILABLE PRODUCTS
=====
ID      Name                Brand                Qty    Price    Origin
-----
1      Anthelois             La Roche-Posay      553    5196    France
2      Nivea SunProtect         Nivea                532    1998    Germany
3      Ultra Sunscreen          Nuetrogena           899    2398    USA
=====

Enter product ID to sell: 1
Enter quantity to sell: 15
Add another item? (yes/no): yes

```

Figure 12 Testing 4 Selling 1st product

Selling second product

```

=====
                          AVAILABLE PRODUCTS
=====
ID      Name                Brand                Qty    Price    Origin
-----
1      Anthelois             La Roche-Posay      533    5196    France
2      Nivea SunProtect         Nivea                532    1998    Germany
3      Ultra Sunscreen          Nuetrogena           899    2398    USA
=====

Enter product ID to sell: 2
Enter quantity to sell: 15
Add another item? (yes/no): no

```

Figure 13 Testing 4 Selling 2nd Product

Bill of both products

```
=====
                                WeCare Wholesale
                                Kamalpokhari, Kathmandu | Phone: 9812345670
                                SALES INVOICE
=====
Customer Name : Krishal
Phone Number  : 9845713016
Date          : 2025-05-14 02:49:03.115789
=====
No  Product Name      Qty  Free  Price  Total
-----
1   Anthelois         15    5   5196   77940
2   Nivea SunProtect   15    5   1998   29970
-----
Subtotal          : Rs. 107910
Grand Total       : Rs. 107910
=====
                                Thank you for shopping with us!
=====
```

Figure 14 Testing 4 Bill of both products

4.5 Test 5 Stock Updates

Objective	To verify stock updates reflect in files
Action	Perform purchase and restock operations
Expected Result	Product.txt shows correct updated quantities
Actual Result	File quantities match program state
Conclusion	The test was successful

Table 5 Testing 5 Stock Updates

```
Anthelois      ,La Roche-Posay,533, 2598, France
Nivea SunProtect,Nivea      ,512, 999 , Germany
Ultra Sunscreen ,Nuetrogena  ,895, 1199, USA
```

Figure 15 Testing 5 Before selling product

AVAILABLE PRODUCTS					
ID	Name	Brand	Qty	Price	Origin
1	Anthelois	La Roche-Posay	533	5196	France
2	Nivea SunProtect	Nivea	512	1998	Germany
3	Ultra Sunscreen	Nuetrogena	895	2398	USA

Enter product ID to sell: 1
Enter quantity to sell: 20
Add another item? (yes/no): no

Figure 16 Testing 5 while selling product

After selling product

```
Anthelois      ,La Roche-Posay,507, 2598, France
Nivea SunProtect,Nivea      ,512, 999 , Germany
Ultra Sunscreen ,Nuetrogena  ,895, 1199, USA
```

Figure 17 Testing 5 After Selling product

```

-----
                        MAIN MENU
-----
1. Sell product to customer
2. Restock products from manufacturer
3. Exit the system
-----
Enter your choice (1, 2, or 3): 2

=====
                        AVAILABLE PRODUCTS
=====
ID      Name                Brand                Qty    Price    Origin
-----
1       Anthelois           La Roche-Posay      507    5196    France
2       Nivea SunProtect      Nivea               512    1998    Germany
3       Ultra Sunscreen      Nuetrogena          895    2398    USA
-----

Enter product ID to restock: 1
Enter quantity to add: 20

```

Figure 18 Testing 5 while purchasing product

```

Anthelois      ,La Roche-Posay,527, 2598, France
Nivea SunProtect,Nivea      ,512, 999 , Germany
Ultra Sunscreen ,Nuetrogena  ,895, 1199, USA

```

Figure 19 Testing 5 After purchasing product

5. Conclusion

The WeCare Wholesale Inventory Management System successfully meets all specified requirements. The implementation provides a robust solution for managing product sales and inventory with proper error handling and data persistence. The system demonstrates effective use of Python data structures and file operations to create a functional business application.

6.Appendix

main.py

```
from read import read_product_data
from operations import handle_sale, handle_restock, display_product_table

def main():
    print("\n" * 2)
    print("\t\t\t\t\tWeCare Wholesale")
    print("\t\t\t\t\tKamalpokhari, Kathmandu | Phone No: 9812345670")
    print("\n" + "=" * 90)
    print("\t\t\t\t\tWelcome to the system Admin! I hope you have a good day ahead!")
    print("=" * 90 + "\n")

    products = read_product_data()
    if not products:
        print("No product data found.")
        return

    while True:
        print("\n" + "-" * 60)
        print("MAIN MENU".center(60))
        print("-" * 60)
        print("1. Sell product to customer")
        print("2. Restock products from manufacturer")
        print("3. Exit the system")
        print("-" * 60)

        try:
            choice = int(input("Enter your choice (1, 2, or 3): "))
        except ValueError:
            print("Invalid input. Please enter a number (1, 2, or 3).")
            continue

        if choice == 1:
            handle_sale(products)
        elif choice == 2:
            handle_restock(products)
        elif choice == 3:
            print("\nThank you for using the system. Goodbye Admin!\n")
            break
        else:
            print("Invalid option. Please choose between 1, 2, or 3.")

if __name__ == "__main__":
    main()
```

operations.py

```

from datetime import datetime
from write import write_product_data

def display_product_table(products):
    print("\n" + "=" * 90)
    print("AVAILABLE PRODUCTS".center(90))
    print("=" * 90)
    print("ID\tName\t\t\tBrand\t\tQty\tPrice\tOrigin")
    print("-" * 90)
    for product_id in products:
        name = products[product_id][0]
        brand = products[product_id][1]
        quantity = products[product_id][2]
        price = str(int(products[product_id][3]) * 2)
        origin = products[product_id][4]
        print(str(product_id) + "\t" + name + "\t" + brand + "\t\t" + quantity + "\t" + price + "\t"
+ origin)
    print("-" * 90 + "\n")

def handle_sale(products):
    customer_name = ""
    while customer_name == "":
        customer_name = input("Enter customer name: ")
        if customer_name == "":
            print("Customer name cannot be empty.")

    while True:
        try:
            customer_phone = int(input("Enter customer phone number: "))
            break
        except ValueError:
            print("Invalid phone number. Digits only.")

    sold_items = []
    total_amount = 0

    while True:
        display_product_table(products)
        try:
            product_id = int(input("Enter product ID to sell: "))
            if product_id not in products:
                print("Invalid product ID.")
                continue

```

```
quantity = int(input("Enter quantity to sell: "))
if quantity <= 0:
    print("Quantity must be greater than 0.")
    continue

available = int(products[product_id][2])
unit_price = int(products[product_id][3]) * 2
free_items = quantity // 3
total_required = quantity + free_items

if total_required > available:
    print("Insufficient stock available.")
    continue

products[product_id][2] = str(available - total_required)
total_cost = quantity * unit_price

sold_items.append([product_id, products[product_id][0], quantity, free_items,
unit_price, total_cost])
total_amount = total_amount + total_cost
except ValueError:
    print("Invalid input. Please enter numbers only.")
    continue

more = input("Add another item? (yes/no): ")
if more.lower() != "yes":
    break

shipping_cost = 0
if total_amount < 1000:
    shipping_cost = total_amount * 0.10

grand_total = total_amount + shipping_cost
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = "invoice_" + timestamp + ".txt"
file = open(filename, "w")

print("\n" + "=" * 90)
print("WeCare Wholesale".center(90))
print("Kamalpokhari, Kathmandu | Phone: 9812345670".center(90))
print("SALES INVOICE".center(90))
print("=" * 90)
print("Customer Name : " + customer_name)
print("Phone Number : " + str(customer_phone))
print("Date : " + str(datetime.now()))
print("-" * 90)
```

```

print("No Product Name      Qty Free Price Total")
print("-" * 90)

file.write("=" * 90 + "\n")
file.write("WeCare Wholesale".center(90) + "\n")
file.write("Kamalpokhari, Kathmandu | Phone: 9812345670".center(90) + "\n")
file.write("SALES INVOICE".center(90) + "\n")
file.write("=" * 90 + "\n")
file.write("Customer Name : " + customer_name + "\n")
file.write("Phone Number : " + str(customer_phone) + "\n")
file.write("Date       : " + str(datetime.now()) + "\n")
file.write("-" * 90 + "\n")
file.write("No Product Name      Qty Free Price Total\n")
file.write("-" * 90 + "\n")

count = 1
for item in sold_items:
    row = str(count) + " " + item[1] + " " * (20 - len(item[1])) + str(item[2]) + " " +
str(item[3]) + " " + str(item[4]) + " " + str(item[5])
    print(row)
    file.write(row + "\n")
    count = count + 1

print("-" * 90)
print("Subtotal      : Rs. " + str(total_amount))
if shipping_cost > 0:
    print("Shipping (10%): Rs. " + str(shipping_cost))
print("Grand Total   : Rs. " + str(grand_total))
print("=" * 90)
print("Thank you for shopping with us!".center(90))
print("=" * 90 + "\n")

file.write("-" * 90 + "\n")
file.write("Subtotal      : Rs. " + str(total_amount) + "\n")
if shipping_cost > 0:
    file.write("Shipping (10%): Rs. " + str(shipping_cost) + "\n")
file.write("Grand Total   : Rs. " + str(grand_total) + "\n")
file.write("=" * 90 + "\n")
file.write("Thank you for shopping with us!".center(90) + "\n")
file.write("=" * 90 + "\n")
file.close()

write_product_data(products)
print("Bill has been saved to file: " + filename)

```

```

from datetime import datetime

```

```

from write import write_product_data

def handle_restock(products):
    display_product_table(products)
    try:
        product_id = int(input("Enter product ID to restock: "))
        if product_id not in products:
            print("Invalid product ID.")
            return
        quantity = int(input("Enter quantity to add: "))
        if quantity <= 0:
            print("Quantity must be positive.")
            return

        current_quantity = int(products[product_id][2])
        new_quantity = current_quantity + quantity
        products[product_id][2] = str(new_quantity)
        write_product_data(products)

        # Create restock invoice
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = "restock_" + timestamp + ".txt"
        file = open(filename, "w")

        print("\n" + "=" * 90)
        print("WeCare Wholesale".center(90))
        print("Kamalpokhari, Kathmandu | Phone: 9812345670".center(90))
        print("RESTOCK RECEIPT".center(90))
        print("=" * 90)
        print("Product Restocked:")
        print("-" * 90)
        print("ID   Product Name          Brand          Added   New Stock")
        print("-" * 90)

        name = products[product_id][0]
        brand = products[product_id][1]

        row = str(product_id) + "   " + name + " " * (22 - len(name)) + brand + " " * (18 -
len(brand)) + str(quantity) + "   " + str(new_quantity)
        print(row)
        print("-" * 90)
        print("Stock updated successfully.\n")

        # File writing
        file.write("=" * 90 + "\n")
        file.write("WeCare Wholesale".center(90) + "\n")

```



```

file.write("Kamalpokhari, Kathmandu | Phone: 9812345670".center(90) + "\n")
file.write("RESTOCK RECEIPT".center(90) + "\n")
file.write("=" * 90 + "\n")
file.write("Product Restocked:\n")
file.write("-" * 90 + "\n")
file.write("ID   Product Name           Brand           Qty Added   New Stock\n")
file.write("-" * 90 + "\n")
file.write(row + "\n")
file.write("-" * 90 + "\n")
file.write("Stock updated successfully.\n")
file.close()

```

```

print("Restock receipt saved as:", filename)

```

```

except ValueError:
    print("Invalid input. Numbers only.")

```

read.py

```

def read_product_data(filename="products.txt"):
    products = {}
    try:
        file = open(filename, "r")
        lines = file.readlines()
        product_id = 1
        for line in lines:
            parts = line.split(",")
            if len(parts) == 5:
                products[product_id] = [
                    parts[0].replace("\n", ""),
                    parts[1].replace("\n", ""),
                    parts[2].replace("\n", ""),
                    parts[3].replace("\n", ""),
                    parts[4].replace("\n", "")
                ]
                product_id += 1
        file.close()
    except FileNotFoundError:
        print("Error: Product file not found.")
    except Exception as error:
        print("An unexpected error occurred: " + str(error))
    return products

```

write.py

```
def write_product_data(products, filename="products.txt"):
    try:
        file = open(filename, "w")
        for product_id in products:
            line = products[product_id][0] + "," + products[product_id][1] + "," +
products[product_id][2] + "," + products[product_id][3] + "," + products[product_id][4] +
"\n"
            file.write(line)
        file.close()
    except Exception as error:
        print("Error writing to file: " + str(error))
```

Bibliography